



A Comprehensive Research Report on Fast and Accurate Minimum Volume Oriented Bounding Boxes for 3D Bin Packing

This report provides a comprehensive analysis of strategies for creating minimum volume oriented bounding boxes (OBBs) for random 3D objects, with a focus on the requirements of speed, accuracy, and visualization. The research synthesizes information from academic literature and software development contexts to provide actionable guidance for implementing an organized 3D bin packing strategy. The core challenge is addressed by exploring algorithmic foundations, practical computational methods using modern libraries like Open3D and Trimesh, data preparation techniques for real-world models, and performance optimization strategies.

Foundational Algorithms for Minimum-Volume Bounding Box Computation

The problem of finding the smallest-volume enclosing box for a set of 3D points or a mesh is a fundamental task in computational geometry with direct applications in 3D bin packing, object pose estimation, and computer graphics ³⁸. The approach to this problem has evolved significantly, moving from exact but computationally prohibitive solutions to highly efficient and accurate approximation algorithms. Understanding these foundational methods is critical for selecting the appropriate tool for a given application, especially when balancing the need for speed against the requirement for minimal volume.

The most well-known exact algorithm was proposed by Joseph O'Rourke in 1985 ^{2 4}. This cubic-time algorithm relies on a key geometric insight: for the minimum-volume bounding box of a convex polyhedron, at least two adjacent faces of the box must each contain an edge of the polyhedron's convex hull, and the remaining four faces must each contain at least one point from the hull ². While theoretically sound, O'Rourke's algorithm has a time complexity that makes it impractical for large datasets common in modern 3D scanning and modeling ¹. Furthermore, its implementation is notoriously difficult, which has limited its widespread adoption ⁵.

To overcome the limitations of exact methods, researchers have developed sophisticated approximation techniques. A significant breakthrough came from the formulation of the problem as an unconstrained optimization on the rotation group $SO(3, \mathbb{R})$, which represents all possible 3D rotations ^{1 5}. This approach, implemented in the Computational Geometry Algorithms Library (CGAL), solves for the optimal rotation that minimizes the bounding box volume ¹. It cleverly combines the Nelder-Mead simplex method—a derivative-free optimization technique—with a metaheuristic inspired by biological evolution. This hybrid strategy effectively balances local search (to refine a solution) and global search (to avoid getting trapped in suboptimal solutions), making it both faster and more reliable than previous techniques ^{1 5}. The ApproxMVBB library offers a

similar C++11 implementation based on sampling and optimization on SO(3), demonstrating excellent performance on very large point clouds ⁶.

For cases where absolute precision is not required, even faster alternatives exist. One such method involves Principal Component Analysis (PCA). By computing the covariance matrix of the point set and finding its eigenvectors, PCA identifies the primary axes of variation in the data ^{25 31}. These eigenvectors define the orientation of an OBB, while the extents are determined by projecting all points onto these axes and finding the resulting minima and maxima ^{25 31}. This PCA-based method is extremely fast and is often used as a robust initial guess or a standalone approximation ^{13 14}. However, its drawback is that it does not guarantee a minimal volume; for example, it can produce suboptimal results for symmetric objects because the choice of eigenvectors for equal eigenvalues is arbitrary ⁴⁸.

The following table summarizes the characteristics of various bounding box computation methods discussed in the provided sources.

Method	Algorithm/ Principle	Key Feature(s)	Time Complexity	Accuracy vs. Speed	Notable Implementations/T
O'Rourke	Exact geometric algorithm	Guaranteed minimal volume; uses convex hull properties ²	Cubic ($O(n^3)$) ¹	Highest Accuracy	Academic publication, hard t
CGAL / ApproxMVBB	Optimization on SO(3)	Formulates problem as continuous optimization; hybrid evolutionary/ Nelder-Mead ^{1 5}	Faster and more reliable than O'Rourke ⁵	High Accuracy, Good Speed	CGAL, ApproxMVBB C++
PCA-based	Principal Component Analysis	Extremely fast; aligns box with data's principal axes ^{25 31}	Low (e.g., $O(n \log n)$ for SVD) ⁷	Approximation, may not be minimal volume ⁴⁸	Open3D, JAMA (Java), custo ²⁵
Edge- Collinearity	Heuristic based on convex hull edges	Ensures at least one convex hull edge is collinear with	Moderate (depends on convex hull size)	High Accuracy, Slower than PCA	Open3D (<code>get_minimal_orient</code> ^{17 19}

Method	Algorithm/ Principle	Key Feature(s)	Time Complexity	Accuracy vs. Speed	Notable Implementations/T
		a box edge ¹⁷ ³³			
Brute Force	Checks all possible orientations	Simple concept	Very high (exponential)	Minimal Volume (if exhaustive)	Mentioned as a reference, no

A more recent and highly optimized method, **MINIMAL_JYLANKI**, is available in Open3D's Tensor API ²¹. This algorithm is described as an exact method inspired by work from Jukka Jylänki, offering a high degree of accuracy, albeit at a slower processing speed compared to approximations ²¹. Its existence within a high-performance library like Open3D makes it a powerful option for offline processing where optimality is paramount. The analytical insight here is that there is no single "best" algorithm; the choice depends on a trade-off between accuracy, speed, and implementation complexity. For a real-time bin packing system, a fast approximation like PCA or a specialized edge-collinearity check is necessary. For generating a precise, static bounding box for a final packed arrangement, an exact method like **MINIMAL_JYLANKI** would be superior.

Practical Implementation Using Modern Computational Libraries

Translating theoretical algorithms into practical code requires leveraging robust, feature-rich computational libraries. Among the most prominent tools for 3D geometry processing are Open3D, Trimesh, and VTK. These libraries abstract away much of the underlying mathematical complexity, providing high-level APIs for loading, manipulating, and analyzing 3D models, including the computation of bounding volumes. They are instrumental in bridging the gap between academic research and real-world applications like 3D bin packing.

Open3D stands out as a particularly relevant library due to its integrated approach to geometry processing and machine learning. It provides several methods for computing oriented bounding boxes directly on its core data structures, such as **PointCloud** and **TriangleMesh**. For users prioritizing speed, Open3D offers a PCA-based approximation via the `get_oriented_bounding_box()` or `create_from_points()` factory method ^{13 14 17}. This method is computationally inexpensive and serves as a good baseline. For higher accuracy, Open3D implements a more rigorous algorithm through the `get_minimal_oriented_bounding_box()` method ^{17 19}. This function operates on the convex hull of the input geometry and intelligently tests potential orientations by ensuring that at least one edge of the convex hull is made collinear with an edge of the candidate bounding box, thereby converging towards the true minimum volume ^{17 33}. The library further enhances this capability with multiple levels of approximation, allowing developers to fine-tune the speed-accuracy trade-off. The **MINIMAL_APPROX** method, for instance, evaluates minimal axis-aligned boxes within the coordinate frame of each triangle in the convex hull and selects the best result, offering a balance between performance and quality ²¹. Finally, for maximum precision, the **MINIMAL_JYLANKI** method can be invoked via the tensor-based API, which is designed for performance and accuracy ²¹.

Trimesh is another powerful Python library that excels in handling triangular meshes. It provides a `mesh.convex_hull` method to compute the convex hull of a mesh ⁵⁴. While its own hull computation can sometimes produce non-watertight results due to numerical precision issues, Trimesh offers valuable repair functions like `fill_holes()` and `fix_normals()` to prepare geometries for analysis ^{42 54}. A key strength of Trimesh is its seamless integration with other scientific Python libraries and, critically, with Open3D. The `mesh.as_open3d` property allows a Trimesh object to be converted into an Open3D object, unlocking access to Open3D's full suite of bounding box algorithms, including its highly accurate `MINIMAL_JYLANKE` method ⁵⁴. This interoperability makes Trimesh a versatile front-end for mesh manipulation, with Open3D serving as a high-performance back-end for specific, intensive tasks like minimal OBB calculation.

Other libraries also contribute to the ecosystem. VTK's `vtkAppendPolyData` and `vtkCleanPolyData` can be used to merge and clean meshes, which is a crucial preprocessing step before surface reconstruction or bounding box computation ⁴⁰. The `vtkFillHolesFilter` is specifically designed to make meshes watertight, a prerequisite for some geometric calculations ⁴⁰. MeshLab is a standalone application that provides a graphical interface for such tasks and includes a filter to compute geometric measures, including bounding box dimensions, directly on a loaded mesh ³⁷. Similarly, command-line utilities like `admesh` and ParaView can read STL files and output bounding box statistics, useful for scripting and batch processing ⁵¹. For visualization purposes, Three.js is a popular WebGL library that can load STL files and compute their bounding boxes on the client side for interactive previews ⁴⁵. These diverse tools demonstrate a rich ecosystem where different libraries can be combined to build a complete workflow, from raw data import to final visualization.

Preparing Random 3D Objects for Bounding Box Analysis

The process of creating a tight-fitting bounding box begins long before the algorithm runs. The quality and structure of the input 3D object, typically represented as a triangle mesh in formats like STL, heavily influence the accuracy and reliability of the resulting bounding box. Real-world 3D scans and models are rarely perfect; they often contain defects such as inverted normals, holes, non-manifold edges, overlapping triangles, and self-intersections ¹². These imperfections can lead to catastrophic failures in geometric computations, including convex hull generation, which is a critical preprocessing step for many minimum-volume bounding box algorithms ⁵³. Therefore, a dedicated and robust data preparation pipeline is essential for any serious implementation.

The first and most critical step is to ensure the mesh is "watertight." A watertight mesh, also known as a closed manifold, has no boundaries (holes) and every edge is shared by exactly two triangles. This property is necessary because algorithms like convex hull computation rely on the ability to traverse the surface unambiguously. Many standard mesh repair operations aim to achieve this state. In practice, a multi-step approach is often required. For instance, one might use `vtkCleanPolyData` in VTK to remove duplicate points and `vtkFillHolesFilter` to seal small gaps, as demonstrated in one workflow ⁴⁰. However, automated hole-filling can fail on complex geometries or leave behind subtle defects. Open3D provides a `fill_holes()` method, but users have reported that it may not work for all cases, and there can be caching issues where the mesh appears repaired but is still not considered watertight by subsequent checks ^{34 42}. In such

scenarios, manual intervention using a dedicated mesh editor like MeshLab or Blender may be unavoidable ⁴⁷.

Once the mesh is cleaned, the next preprocessing step is to compute its convex hull. The convex hull is the smallest convex set that contains the original shape, and a vital theorem in this domain states that the minimum bounding box of a point set is identical to that of its convex hull ⁴. Computing the convex hull reduces the complexity of the problem by discarding all interior points, leaving only the defining outer shell. Open3D's `compute_convex_hull()` method, which uses the Qhull library, is a standard tool for this task and is supported for both `PointCloud` and `TriangleMesh` objects ^{16 32}. However, the convex hull itself can introduce new problems. Numerical precision errors during the computation can lead to a non-manifold or non-watertight convex hull, even if the original mesh was valid ⁵³. This is a known issue in libraries that rely on `scipy/qhull` for their convex hull functionality ⁵³. To mitigate this, some advanced methods incorporate pre-processing steps before convex hull computation. One such technique involves removing "chamfer" faces that form near sharp edges during the hull construction, which can reduce the number of faces by nearly 50% without sacrificing accuracy, thereby speeding up subsequent calculations ⁹.

Another important consideration is the density and distribution of the input data. For point clouds, uniformity and sufficient sampling are key. If the point cloud is too sparse in certain areas, the resulting bounding box may not accurately represent the object's true extent. Some algorithms, like Poisson surface reconstruction, offer parameters to control the level of detail and can help generate a better-behaved mesh from a noisy or incomplete point cloud ⁴³. Open3D's `simplify_vertex_clustering` method can be used to downsample a dense mesh into a coarser representation that retains the overall shape, which can be beneficial for speeding up later processing ^{35 39}. The decision to simplify should be guided by the trade-off between computational speed and the fidelity of the bounding box to the original model. Ultimately, the preparation stage is not a minor housekeeping task but a critical phase where the foundation for accurate and efficient bounding box computation is laid.

Achieving Visual Integration: Rendering STL Objects Inside Their Bounding Box

A core component of the user's request is the desire to visualize the STL object inside its computed bounding box. This visual feedback is invaluable for debugging, validation, and understanding the effectiveness of the packing strategy. Fortunately, the modern computational geometry and 3D rendering ecosystems provide mature and powerful tools to accomplish this with relative ease. The process generally involves loading the 3D models, performing the geometric computation to find the bounding box, and then rendering both the object and the box together in a unified scene.

The visualization capabilities are deeply integrated into libraries like Open3D and Three.js. Open3D provides a high-level `draw_geometries` function that simplifies the creation of a visualization window ³². To display an object and its bounding box, one simply adds both geometries (the mesh and the box) to a list and passes it to this function. The library handles the rendering setup, including setting up a camera and lighting. For more customized or programmatic control, Open3D supports a more advanced web-based visualizer introduced in version 0.13.0 ^{20 23}. This web visualizer allows

embedding 3D scenes in environments like Jupyter notebooks and standalone web applications, and it supports advanced features like physically-based rendering (PBR) materials and multiple lighting systems, enabling the creation of high-quality renderings ²⁰. Visualization can also be done by adding the geometries to a viewer object and calling its `run()` method ⁴⁴. In these visualizations, the bounding box can be styled independently, for example, by setting its color to red `[1, 0, 0]` to make it stand out from the object ⁴⁴.

The Trimesh library offers a similar, more Python-centric approach to visualization. Its `scene.show()` method can render a mesh along with any transformations applied to it, which could include displaying the bounding box as a separate geometric entity ⁵⁴. The underlying principle remains the same across these libraries: the object and the bounding box are treated as distinct geometric entities within a shared coordinate system.

On the web, the process is handled by JavaScript libraries built on top of WebGL, such as Three.js. A typical workflow involves setting up a `THREE.Scene`, a `THREE.PerspectiveCamera`, and a `THREE.WebGLRenderer` ⁴⁵. An STL file can be loaded into the scene using a loader like `STLLoader` ⁴⁵. Once the model is loaded, its bounding box can be calculated using `THREE.Box3().setFromObject(mesh)` to get its dimensions and position ⁴⁵. To visualize the box, its eight corner points can be extracted and used to define the edges of a wireframe cube, which is then added to the scene ²⁸. This allows for real-time interaction, such as rotating and zooming around the object and its box, using controls like `OrbitControls` ⁴⁵. Foxglove 3D is another tool that supports direct visualization of STL files and can render meshes with various color modes, though it does not automatically overlay bounding boxes, requiring custom scripts to reconstruct and display them ⁴⁹.

The analytical insight here is that the technology for visualization is readily available and relatively straightforward to integrate. The primary challenge is not in the rendering itself but in correctly computing and representing the bounding box's transformation (its center, rotation, and dimensions) so that it aligns perfectly with the object. Once this geometric relationship is established—typically by storing the bounding box's center, rotation matrix, and extent vector—the rest of the process is a matter of using the chosen library's API to draw these components. The choice of visualization tool should be guided by the project's deployment context: for rapid prototyping and analysis, a library like Open3D is ideal; for interactive web-based previews or integrations, Three.js is the standard.

Performance Optimization Strategies for Real-Time Applications

While the previous sections have focused on accuracy and correctness, the user's explicit requirement for a "fast" solution necessitates a deep dive into performance optimization. Achieving real-time performance, such as maintaining 60 frames per second (fps) when processing hundreds of items, demands a strategic approach that goes beyond simply choosing a fast algorithm. It involves architectural decisions about data structures, parallelization, and a careful management of the accuracy-versus-speed trade-off.

The most direct path to speed is to select a low-complexity algorithm. As established, Principal Component Analysis (PCA) provides a very fast approximation of an oriented bounding box ^{25 31}. Its computational cost is dominated by the Singular Value Decomposition (SVD) of the data matrix, an operation with a manageable time complexity ⁷. For many bin packing applications, the slight

inefficiency of a PCA-based box compared to a minimal-volume box may be an acceptable price to pay for the dramatic increase in speed. Another strategy is to operate on a simplified or downsampled version of the geometry. For a mesh, methods like **simplify_vertex_clustering** in Open3D can drastically reduce the number of vertices and triangles while preserving the overall shape ^{35 39}. For a point cloud, one could randomly sample a fixed number of points or use grid-based clustering to reduce density. This pre-processing step directly reduces the workload for the convex hull and bounding box computation.

Parallelization is another powerful lever for improving performance. Many modern CPUs have multiple cores, and computational geometry algorithms are often amenable to parallel execution. The ApproxMVBB library, for example, explicitly supports multithreading via OpenMP, allowing it to distribute the workload of evaluating different orientations across multiple CPU threads ⁶. Similarly, Open3D's new Neighbor Search module, introduced in version 0.13.0, is GPU-accelerated, which can aid in the fast execution of certain 3D geometric queries ^{20 36}. Leveraging these parallel capabilities can lead to significant speedups, especially when dealing with large point clouds containing millions of points ⁶.

Beyond algorithmic choices, architectural patterns play a crucial role. For dynamic scenes where objects are being packed and repacked continuously, recomputing the bounding box from scratch for every movement is wasteful. Instead, a common pattern is to cache the bounding box information and only update it when necessary—for instance, when an object's rotation changes significantly. When an object is rotated, its cached bounding box can be transformed using the same rotation matrix, which is a very fast operation. The bounding box class in Open3D, for instance, stores its orientation as a 3x3 rotation matrix, which can be directly applied to transform the box ^{13 18}. This avoids redundant and expensive hull and optimization calculations.

Finally, the entire workflow must be profiled to identify actual bottlenecks. A developer might assume that convex hull computation is the slowest part, only to find through profiling that the I/O operations for reading STL files or the memory allocation overhead is the true bottleneck. The context mentions a user concerned about performance in a similar scenario, though their case involved structured rectangles rather than random 3D objects ²⁶. This highlights the importance of empirical measurement over assumptions. By combining a fast algorithm, intelligent data reduction, parallel processing, smart caching, and targeted profiling, it is feasible to build a highly performant 3D bin packing system that meets stringent real-time constraints.

Synthesizing a Strategy for 3D Bin Packing

In synthesizing the findings of this report, a clear, multi-stage strategy emerges for implementing an organized 3D bin packing system that addresses the user's core requirements: creating tight-fitting bounding boxes for random 3D objects, visualizing them, and achieving this as fast as possible. This strategy moves from the general to the specific, outlining a logical workflow that balances accuracy, speed, and robustness.

Stage 1: Data Preparation and Robustness Pipeline The foundation of any successful geometric computation is clean, valid data. The first step is to establish a robust pipeline for preparing 3D models, typically in STL format.

1. **Load and Inspect:** Use a library like Trimesh or Open3D to load the STL file ^{46 54}.
2. **Repair Imperfections:** Implement a sequence of repair operations to handle

common STL errors. This should include fixing normals (`trimesh.util.fix_winding` or `trimesh.Trimesh.fix_normals`), removing duplicate vertices, and filling small holes (`trimesh.util.fill_holes` or `numpy.stl.mesh.Mesh.fill_holes`)^{12 42 54}. For larger or complex holes, consider using a specialized tool like MeshLab or a combination of VTK filters^{40 47}. 3. Ensure Watertightness: After repairs, verify that the mesh is watertight using methods like `mesh.is_watertight` in Trimesh or `is_watertight` in Open3D³⁵. A non-watertight mesh will cause failures in subsequent convex hull computation⁵³.

Stage 2: Bounding Box Computation Strategy With a prepared mesh, the next stage is to choose an appropriate algorithm based on the desired balance of speed and accuracy. * **For Real-Time Performance:** If the application requires processing dozens of objects per second (e.g., for dynamic packing), a fast approximation is mandatory. The recommended approach is to use Open3D's PCA-based method: `mesh.compute_vertex_normals(); o3d_box = mesh.get_oriented_bounding_box()`. This provides a reasonable OBB almost instantaneously^{13 31}. If this proves insufficiently tight, switch to the **MINIMAL_APPROX** method, which offers a better fit by testing orientations aligned with the mesh's face normals²¹. * **For Maximum Accuracy:** If the packing configuration is finalized and a precise, static bounding box is needed, an exact method should be used. The **MINIMAL_JYLANKEI** method in Open3D's tensor API (`open3d.t.geometry.MINIMAL_JYLANKEI`) is the gold standard for accuracy, albeit slower^{13 21}. Alternatively, the convex hull can be exported to Trimesh and used with its `.as_open3d` property to leverage Open3D's minimal box computation⁵⁴.

Stage 3: Visualization and Integration Visualization is crucial for validating the results and integrating the bounding box into the bin packing logic. 1. **Visualize:** Use Open3D's `draw_geometries` or its web-based visualizer to render the original mesh and its computed bounding box side-by-side^{20 32}. The box can be colored distinctly (e.g., red) to differentiate it from the object⁴⁴. 2. **Integrate:** The **OrientedBoundingBox** object in Open3D contains all necessary information: its center (`obox.center`), rotation matrix (`obox.R`), and extent (`obox.extents`)^{13 18}. This data defines the box's position, orientation, and size in 3D space. This representation is ideal for use in a bin packing algorithm, where it can be used to perform collision detection and spatial sorting.

By following this synthesized strategy, one can build a modular and effective system. The data preparation stage ensures resilience against poor-quality inputs. The bounding box computation stage provides flexibility to meet varying performance requirements. And the final integration stage delivers the precise geometric data needed for the core bin packing logic, all while providing clear visual feedback for validation.

Reference

1. New in CGAL: Optimal Bounding Box https://www.cgal.org/2020/04/20/Optimal_bounding_box/
2. Minimum bounding box algorithms - Wikipedia https://en.wikipedia.org/wiki/Minimum_bounding_box_algorithms

3. Any fast and robust implementation to calculate the minimum ... <https://stackoverflow.com/questions/10941718/any-fast-and-robust-implementation-to-calculate-the-minimum-bounding-box-of-a-3d>
4. Minimum bounding box - Wikipedia https://en.wikipedia.org/wiki/Minimum_bounding_box
5. Fast oriented bounding box optimization on the rotation group $SO(3, \mathbb{R})$ <https://dl.acm.org/doi/abs/10.1145/2019627.2019641>
6. gabyx/ApproxMVBB: Fast algorithms to compute an approximation ... <https://github.com/gabyx/ApproxMVBB>
7. Understanding Convex Hull Algorithms: A Comprehensive Guide <https://algorithms.wtf/blog/understanding-convex-hull-algorithms-a-comprehensive-guide/>
8. 3D Convex Hull-Based Registration Method for Point Cloud ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC6695679/>
9. Estimation of minimum volume of bounding box for geometrical ... https://www.metrology-journal.org/articles/ijmqe/full_html/2020/01/ijmqe200007/ijmqe200007.html
10. stl files -measuring stl files - 3d models - 3D Printing Stack Exchange <https://3dprinting.stackexchange.com/questions/2945/stl-files-measuring-stl-files>
11. Efficient Convex-Hull-Based Vehicle Pose Estimation Method for 3D ... <https://journals.sagepub.com/doi/abs/10.1177/03611981241250027>
12. Understanding STL Files for 3D Printing - Fathom Manufacturing <https://fathommfg.com/blog/guide-to-better-stl-files>
13. open3d.geometry.OrientedBoundingBox https://www.open3d.org/docs/latest/python_api/open3d.geometry.OrientedBoundingBox.html
14. open3d.geometry.OrientedBoundingBox https://www.open3d.org/html/python_api/open3d.geometry.OrientedBoundingBox.html
15. Adjusting (center & rotation) of oriented bounding box to best fit points <https://stackoverflow.com/questions/79398514/adjusting-center-rotation-of-oriented-bounding-box-to-best-fit-points>
16. open3d.geometry.TriangleMesh - Open3D 0.19.0 documentation https://www.open3d.org/docs/release/python_api/open3d.geometry.TriangleMesh.html
17. open3d.geometry.PointCloud https://www.open3d.org/docs/latest/python_api/open3d.geometry.PointCloud.html
18. python - How do I get the orientation from a open3d.geometry ... <https://stackoverflow.com/questions/66712854/how-do-i-get-the-orientation-from-a-open3d-geometry-orientedboundingbox>
19. open3d.geometry.OrientedBoundingBox https://www.open3d.org/docs/0.17.0/python_api/open3d.geometry.OrientedBoundingBox.html
20. Category: Uncategorized - Open3D <https://www.open3d.org/category/uncategorized/>

21. `open3d.t.geometry.OrientedBoundingBox` https://www.open3d.org/docs/latest/python_api/open3d.t.geometry.OrientedBoundingBox.html
22. `open3d.geometry.MeshBase` https://www.open3d.org/docs/latest/python_api/open3d.geometry.MeshBase.html
23. A Modern Library for 3D Data Processing - Open3D <https://www.open3d.org/author/administratorivcl-org/>
24. `open3d.geometry.PointCloud` - Open3D 0.19.0 documentation https://www.open3d.org/docs/release/python_api/open3d.geometry.PointCloud.html
25. Find Bounding Box of a 3d point cloud using PCA - Stack Overflow <https://stackoverflow.com/questions/16986025/find-bounding-box-of-a-3d-point-cloud-using-pca>
26. Efficient convex hull around rectangles (and checking if a point lies ... <https://stackoverflow.com/questions/28232526/efficient-convex-hull-around-rectangles-and-checking-if-a-point-lies-within-the>
27. Finding minimum-area-rectangle for given points? <https://gis.stackexchange.com/questions/22895/finding-minimum-area-rectangle-for-given-points>
28. Understanding and Creating the Bounding Box of a Geometry <https://medium.com/@egimata/understanding-and-creating-the-bounding-box-of-a-geometry-d6358a9f7121>
29. Mastering Minimum Bounding Box - Number Analytics <https://www.numberanalytics.com/blog/minimum-bounding-box-computational-geometry>
30. Efficient 3D Convex Hull Tutorial - Codeforces <https://codeforces.com/blog/entry/81768>
31. yudhisteer/Point-Clouds-3D-Perception-with-Open3D - GitHub <https://github.com/yudhisteer/Point-Clouds-3D-Perception-with-Open3D>
32. Point cloud - Open3D primary (unknown) documentation <https://www.open3d.org/docs/latest/tutorial/geometry/pointcloud.html>
33. `open3d.geometry.TriangleMesh` https://www.open3d.org/docs/latest/python_api/open3d.geometry.TriangleMesh.html
34. Struggling to create a suitable watertight mesh using Open3D from ... <https://github.com/isl-org/Open3D/discussions/3913>
35. Mesh - Open3D primary (unknown) documentation <https://www.open3d.org/docs/latest/tutorial/geometry/mesh.html>
36. Blog - Open3D <https://www.open3d.org/blog/>
37. List of Filters — PyMeshLab documentation - Read the Docs https://pymeshlab.readthedocs.io/en/2021.10/filter_list.html
38. Mastering Minimum Bounding Box - Number Analytics <https://www.numberanalytics.com/blog/ultimate-guide-minimum-bounding-box-computational-geometry>

39. Mesh — Open3D latest (664eff5) documentation <https://www.open3d.org/docs/latest/tutorial/Basic/mesh.html>
40. python - how to make an open stl file watertight - Stack Overflow <https://stackoverflow.com/questions/64804792/how-to-make-an-open-stl-file-watertight>
41. How to clean up the mesh in Open3D? - Stack Overflow <https://stackoverflow.com/questions/67466335/how-to-clean-up-the-mesh-in-open3d>
42. How to fix non-watertight model? #509 - mikedh/trimesh - GitHub <https://github.com/mikedh/trimesh/issues/509>
43. Surface Reconstruction — Open3D latest (664eff5) documentation https://www.open3d.org/docs/latest/tutorial/Advanced/surface_reconstruction.html
44. How to plot 3D bounding boxes on Point Cloud Data(PCD) files for ... <https://stackoverflow.com/questions/60770443/how-to-plot-3d-bounding-boxes-on-point-cloud-datapcd-files-for-visualization>
45. How to Use WebGL for 3D Printing Previews on the Web <https://blog.pixelfreestudio.com/how-to-use-webgl-for-3d-printing-previews-on-the-web/>
46. Blog – Page 2 - Open3D <https://www.open3d.org/blog/page/2/>
47. Complete a partial mesh and make it watertight - Stack Overflow <https://stackoverflow.com/questions/65065925/complete-a-partial-mesh-and-make-it-watertight>
48. OBB Computation - General and Gameplay Programming <https://www.gamedev.net/forums/topic/608086-obb-computation/>
49. 3D panel | Foxglove Docs <https://docs.foxglove.dev/docs/visualization/panels/3d>
50. Visualizing the 3D bounding boxes created by BasicWriter <https://forums.developer.nvidia.com/t/visualizing-the-3d-bounding-boxes-created-by-basicwriter/241978>
51. [surface handling] Command line tool for finding the bounding box of ... <https://www.cfd-online.com/Forums/openfoam-meshing/61514-command-line-tool-finding-bounding-box-stl-file.html>
52. Segmentations - 3D Slicer documentation - Read the Docs https://slicer.readthedocs.io/en/latest/user_guide/modules/segmentations.html
53. Non watertight convex hull · Issue #535 · mikedh/trimesh - GitHub <https://github.com/mikedh/trimesh/issues/535>
54. trimesh — Tidy3D Electromagnetic Solver https://docs.flexcompute.com/projects/tidy3d/en/v2.7.2/api/_autosummary/trimesh.html
55. libigl tutorial <https://libigl.github.io/tutorial/>